Universidad Autónoma de Entre Ríos — Facultad de Ciencia y Tecnología
Licenciatura en Sistemas de Información — Sistemas Operativos / Sistemas Distribuidos

ShopResolver

Caso práctico 4: Nombrado en Sistemas Distribuidos

Alumno	Julián Olivera
Docente	Ing. Mg. Ulises Rapallini
Repositorio	https://github.com/julianAO2002/shopresolver
Fecha	11/11/2026

Descripción del sistema

Plataforma de comercio electrónico mínima para estudiar **nombrado en sistemas distribuidos**: un `OrdersService` que consulta a un `InventoryService` usando solo su nombre lógico (`http://inventory-service:8080`), nunca su dirección IP.

Caso práctico 4 — Sistemas Operativos / Sistemas Distribuidos (FCyT – UADER). Autor: Julián Olivera.

Contenido

Ruta	Qué hay
`InventoryService/`	Web API en C# (.NET 8). `GET /api/inventory/{productId}` devuelve el stock, el nombre del contenedor (`server`) y su IP (`address`).
`OrdersService/`	Web API en C# (.NET 8). Consulta a inventario por nombre con `HttpClient`.
`docker-compose.yml`	Ambos servicios en la red `ecommerce-network`.
`k8s/shopresolver.yaml`	El mismo sistema en Kubernetes: 3 Pods de inventario detrás de un `Service`.
`scripts/`	Un script por grupo de ejercicios; cada uno guarda su salida en `docs/salidas/`.
`docs/salidas/`	Salidas reales usadas como evidencia en este README y en el PDF.
`docs/generar_pdf.py`	Genera el PDF a partir de este README y de las salidas.

Requisitos

- Docker Desktop (o Docker Engine + Compose v2).
- Bash (en Windows sirve Git Bash) para los scripts.
- Solo para el ejercicio 13: `kind` y `kubectl`.
- No hace falta tener .NET instalado: las imágenes compilan con `mcr.microsoft.com/dotnet/sdk:8.0`.

Ejecución rápida

```
docker compose up -d --build
curl http://localhost:8081/api/orders/inventory/P001
curl http://localhost:8081/api/orders/resolve
curl -X POST http://localhost:8081/api/orders -H "content-type: application/json" -d '{"productId":"P001","quan
tity":2}'
docker compose down
```

Endpoints de `OrdersService` (puerto 8081 del host):

Método y ruta	Qué hace	Respuestas
`GET /api/orders/inventory/{productId}`	Llama a `http://inventory-service:8080/api/inventory/{productId}` y devuelve la URL usada y la respuesta	200, 404, 503 si inventario no responde
`GET /api/orders/resolve?host=`	Resuelve un nombre desde dentro del contenedor (por defecto `inventory-service`)	200, 404
`POST /api/orders`	Crea un pedido si hay stock (`{"productId":"P001","quantity":2}`)	201, 400, 409 sin stock, 422 producto inexistente, 503
`GET /api/orders`	Lista los pedidos	200

Productos precargados: `P001` (25), `P002` (8), `P003` (0).

La URL de inventario se toma de la variable `Inventory__BaseUrl` (por defecto `http://inventory-service:8080`). Así se puede comparar el nombre lógico contra una IP fija sin tocar el código (ejercicio 7).

Reproducir las evidencias:

```
bash scripts/ej01-05_docker_dns.sh   # red Docker, DNS embebido, cambio de IP, DNS de Internet
bash scripts/ej07-11_compose.sh      # C# en Compose, IP fija vs nombre, recreación, 3 instancias
bash scripts/ej13_kubernetes.sh      # Service de Kubernetes en un cluster kind
```

Respuestas

1. Preguntas teóricas

Nombre, identificador y dirección

Nombre: `inventory-service` — Dirección: `172.18.0.5`

1. ¿Qué diferencia existe entre el nombre `inventory-service` y la dirección `172.18.0.5`? `inventory-service` es un nombre lógico, legible y estable que designa *qué* servicio se quiere usar. `172.18.0.5` es una dirección: indica *dónde* está, en ese momento, el punto de acceso (una interfaz de red de un contenedor concreto). El nombre se mantiene aunque el contenedor se recree; la dirección no: en la práctica, al recrear el contenedor pasó de `172.30.0.2` a `172.30.0.4` y el nombre siguió funcionando (ejercicio 4).

2. ¿Qué se entiende por nombre dentro de un sistema distribuido? Una cadena de bits o caracteres que se usa para referirse a una entidad (un host, un servicio, un archivo, un usuario, un proceso). Los nombres “amigables” (*human-friendly*) están pensados para que los usen personas y programadores, y deben resolverse para poder acceder a la entidad.

3. ¿Qué se entiende por identificador? Un nombre con tres propiedades (Tanenbaum y Van Steen): referencia a lo sumo a una entidad, cada entidad tiene a lo sumo un identificador y nunca se reutiliza para otra entidad, ni siquiera cuando la original deja de existir. Ejemplos: el ID del contenedor que asigna Docker (`d85808269c04…`), el UID de un Pod de Kubernetes, un UUID o una dirección MAC.

4. ¿Qué se entiende por dirección? El nombre de un punto de acceso (*access point*) a una entidad: lo que se necesita para comunicarse con ella efectivamente, por ejemplo IP + puerto (`172.18.0.5:8080`). Una entidad puede tener varios puntos de acceso y estos pueden cambiar con el tiempo o ser reasignados a otra entidad.

5. ¿Una dirección puede utilizarse como identificador? Justifique. En general no. Una dirección no cumple las propiedades de un identificador: puede cambiar aunque la entidad sea la misma (recrear el contenedor cambia la IP), puede reasignarse a otra entidad (en el ejercicio 7 la IP `172.29.0.2` que tenía inventario pasó a ser de otro contenedor, `filler`) y una entidad puede tener varias direcciones (un contenedor conectado a dos redes). Usarla como identificador lleva a referencias obsoletas o, peor, a hablar con la entidad equivocada.

6. ¿Puede una misma entidad tener diferentes nombres? Proporcione un ejemplo. Sí, se llaman alias. En Docker el mismo contenedor responde a `inventory-service`, a su ID corto (`d85808269c04`, visible en `DNSNames`) y, en Compose, también a `shopresolver-inventory-service-1`. En Kubernetes el Service responde a `inventory-service`, `inventory-service.ecommerce` y `inventory-service.ecommerce.svc.cluster.local`. En DNS, un registro `CNAME` hace que `www.empresa.com` sea alias de `empresa.com`.

7. ¿Puede un mismo nombre representar diferentes instancias de un servicio? Sí. El nombre identifica al servicio, no a la réplica. En el ejercicio 11, `inventory-service` resolvió a tres direcciones (`172.29.0.2`, `.4` y `.5`), una por instancia, y las consultas se repartieron entre ellas. Es la base del escalado horizontal y del balanceo de carga.

Resolución de nombres

1. ¿Qué significa resolver un nombre? Obtener, a partir del nombre, la información necesaria para acceder a la entidad: en general su dirección. Es una búsqueda en un servicio de nombres (DNS, el DNS embebido de Docker, CoreDNS en Kubernetes, un registro de servicios).

2. ¿Qué componente transforma `inventory-service` en una dirección? El servidor DNS embebido de Docker, que escucha en `127.0.0.11` dentro de cada contenedor conectado a una red definida por el usuario (se ve en `/etc/resolv.conf`: `nameserver 127.0.0.11`). El resolver de la libc del contenedor le pregunta y Docker responde con la IP actual del contenedor llamado así. En Kubernetes ese rol lo cumple CoreDNS (`nameserver 10.96.0.10`).

3. ¿Por qué conviene utilizar nombres en lugar de IP configuradas directamente? Porque desacopla al cliente de la ubicación del servidor: las IP las asigna dinámicamente la plataforma y cambian al recrear, reiniciar, mover o escalar. Con un nombre la configuración no cambia, es legible, permite tener varias instancias detrás y evita reconfigurar y redesplegar clientes. Con una IP fija, el `OrdersService` del ejercicio 7 quedó apuntando a una dirección obsoleta y devolvió 503.

4. ¿Qué ocurre si `inventory-service` cambia su dirección IP? El DNS de Docker actualiza el registro automáticamente. La siguiente resolución devuelve la IP nueva y el cliente sigue funcionando sin modificaciones. Si el cliente hubiera guardado la IP vieja, sus pedidos fallarían (conexión rechazada o *timeout*) o llegarían a otro contenedor que heredó esa IP.

5. ¿Qué relación existe entre resolución de nombres y transparencia de ubicación? La resolución de nombres es el mecanismo que hace posible la transparencia de ubicación: el cliente usa un nombre que no dice nada sobre dónde está el recurso, y la traducción a una dirección concreta ocurre en tiempo de ejecución, fuera del cliente. Si además el recurso puede moverse sin que el cliente lo note, se obtiene transparencia de migración/reubicación.

Espacios de nombres

Considere: `api.inventory.empresa.com`

1. ¿Qué es un espacio de nombres? El conjunto organizado de todos los nombres válidos de un sistema y las reglas para formarlos. Se representa como un grafo de nombres con nodos hoja (entidades) y nodos

directorio (contextos que contienen otros nombres). Dentro de un mismo contexto un nombre es único.

2. ¿Corresponde a un esquema plano o jerárquico? Jerárquico. El nombre está formado por etiquetas separadas por puntos, y cada una es un nivel de un árbol.

3. Identifique los diferentes niveles del nombre. Se lee de derecha a izquierda, desde la raíz:

Nivel	Etiqueta	Significado
0	`.` (implícito)	Raíz del DNS
1	`com`	Dominio de nivel superior (TLD)
2	`empresa`	Dominio de la organización (`empresa.com`)
3	`inventory`	Subdominio del área o sistema de inventario
4	`api`	Host / servicio concreto (`api.inventory.empresa.com`)

4. ¿Qué ventajas presenta un espacio de nombres jerárquico?

- Unicidad global sin coordinación central: cada contexto solo garantiza que sus nombres sean únicos dentro de él.
- Delegación de autoridad: `empresa` administra todo lo que cuelga de `empresa.com`.
- Escalabilidad: la base de datos se reparte en zonas y servidores.
- Resolución eficiente por partes (iterativa o recursiva) y caché por niveles.
- Nombres legibles que reflejan la organización.

5. ¿Por qué DNS utiliza una estructura jerárquica? Porque tiene que nombrar cientos de millones de hosts administrados por organizaciones independientes. La jerarquía permite partir el espacio en zonas, delegar cada zona a quien la administra, distribuir la carga entre muchos servidores y cachear las respuestas de los niveles altos (que casi no cambian). Un espacio plano necesitaría una autoridad única y no escalaría.

DNS

1. ¿Qué función cumple DNS? Es el servicio de nombres de Internet: traduce nombres de dominio a información asociada, sobre todo direcciones IP (registros `A`/`AAAA`), pero también servidores de correo (`MX`), alias (`CNAME`), servidores de nombres (`NS`) y texto (`TXT`).

2. ¿Por qué DNS puede considerarse un sistema distribuido? Porque su base de datos está repartida entre millones de servidores autónomos de distintas organizaciones (raíz, TLD, autoritativos, recursivos), que cooperan mediante un protocolo común para resolver cualquier nombre, con replicación (servidores primarios y secundarios, *anycast*), caché en todos los niveles y sin un punto central que conozca todo. Para el usuario se comporta como un único servicio.

3. ¿Qué información almacena un servidor DNS? Registros de recursos (*resource records*) de las zonas de las que es autoridad: `SOA`, `NS`, `A`, `AAAA`, `CNAME`, `MX`, `PTR`, `TXT`, `SRV`, cada uno con su TTL. Un servidor recursivo, además, guarda en caché respuestas obtenidas de otros servidores mientras dure su TTL.

4. ¿Qué diferencia existe entre un nombre de dominio y una dirección IP? El nombre de dominio es un identificador lógico, jerárquico y legible (`www.google.com`) que no depende de la red física. La IP es la dirección numérica que usa la capa de red para enrutar paquetes. Un nombre puede apuntar a muchas IP (en el ejercicio 5, `www.google.com` devolvió 8 direcciones IPv4 y 8 IPv6) y las IP de un nombre pueden cambiar sin cambiar el nombre.

5. ¿Qué ventajas proporciona el uso de caché? Menor latencia (la respuesta está cerca), menos carga sobre servidores raíz, TLD y autoritativos, menos tráfico en la red y cierta tolerancia a fallas temporales de los

servidores autoritativos. El costo es la consistencia: un cambio puede tardar hasta el TTL en verse (por eso Docker usa un TTL de 600 s pero responde siempre con su estado actual, y Google usa TTL de 300 s).

6. ¿Qué ocurriría si las aplicaciones tuvieran que conocer directamente las IP de todos los servicios? Cada cambio de dirección (migración, reinicio, cambio de proveedor, escalado, falla) obligaría a reconfigurar y redistribuir todas las aplicaciones. Habría configuraciones desactualizadas, errores, imposibilidad de balancear carga o de mover servicios sin cortes, y un acoplamiento total entre clientes e infraestructura. En la práctica, Internet y cualquier sistema dinámico serían inmanejables.

Nombrado en arquitecturas modernas

1. ¿Por qué resulta inconveniente configurar manualmente las IP de los microservicios? Porque en microservicios las instancias son efímeras: se crean, destruyen, reinician y escalan constantemente y la plataforma les asigna IP dinámicas. Una IP configurada a mano queda obsoleta en el primer redespliegue, no admite varias réplicas, no se adapta a fallas y obliga a coordinar cambios de configuración entre equipos.

2. ¿Qué ocurre cuando un contenedor es eliminado y creado nuevamente? Es un contenedor nuevo: recibe otro ID y, en general, otra dirección IP (la que el IPAM de Docker tenga libre). Lo único que se conserva es el nombre, porque se vuelve a crear con el mismo `--name` o porque Compose usa el mismo nombre de servicio. En el ejercicio 4 la IP pasó de `172.30.0.2` a `172.30.0.4`.

3. ¿Por qué un nombre puede permanecer estable aunque cambie la dirección? Porque el nombre y la dirección están desacoplados por una tabla de asociación (*binding*) que mantiene el servicio de nombres. Cuando la entidad cambia de ubicación solo se actualiza esa asociación; el nombre, que es lo que conocen los clientes, no cambia.

4. ¿Qué se entiende por Service Discovery? El mecanismo por el cual un cliente encuentra, en tiempo de ejecución, las instancias disponibles de un servicio a partir de su nombre lógico. Incluye un registro (*service registry*) con las instancias vivas, la forma en que estas se registran y dan de baja, controles de salud y la consulta (por DNS o por API). Ejemplos: DNS embebido de Docker, Services + CoreDNS de Kubernetes, Consul, Eureka.

5. ¿Qué relación existe entre Service Discovery y nombrado? Service Discovery es un servicio de nombres especializado para entornos dinámicos: resuelve un nombre lógico de servicio a un conjunto de direcciones de instancias sanas que cambia constantemente. El nombre es la clave de búsqueda; el registro es el espacio de nombres.

6. ¿Qué diferencias existen entre DNS tradicional y descubrimiento de servicios?

Aspecto	DNS tradicional	Service Discovery
Dinamismo	Registros relativamente estables, cambios manuales o poco frecuentes	Instancias que aparecen y desaparecen todo el tiempo, registro automático
Caché / TTL	TTL de minutos u horas, propagación lenta	TTL cortos o nulos, información casi en tiempo real
Salud	No sabe si el host responde	Health checks: solo devuelve instancias sanas
Información	Nombre → IP (y algunos registros extra)	Nombre → instancias con IP, puerto, versión, metadatos
Balanceo	Round-robin básico	Balanceo del lado del cliente o del servidor (proxy, ClusterIP)
Alcance	Global, jerárquico, multi-organización	Interno a un cluster o plataforma

---

2. Ejercicios

Caso: `orders-service` usaba `http://172.18.0.5:8080/api/inventory`; al recrear el contenedor, inventario recibió `172.18.0.9` y el pedido se rompió. La propuesta es usar `http://inventory-service:8080/api/inventory`. Los ejercicios lo comprueban.

> Las IP de las salidas son de las redes que creó Docker en mi máquina (`172.30.0.0/16` en los ejercicios 1 a 5 y `172.29.0.0/16` con Compose); en otra máquina pueden ser `172.18.x.x` u otras. Las salidas completas están en `docs/salidas/`.

Ejercicio 1 — Crear una red Docker

```
$ docker network create ecommerce-network
$ docker network ls --filter name=ecommerce-network
NETWORK ID     NAME                DRIVER    SCOPE
37ddea68d0dc   ecommerce-network   bridge    local
$ docker network inspect ecommerce-network
Name=ecommerce-network Driver=bridge Subnet=172.30.0.0/16 Gateway=172.30.0.1
```

1. **¿Qué función cumple una red Docker?** Crea una red virtual aislada (un *bridge* de Linux con su subred, `172.30.0.0/16`, y gateway `172.30.0.1`) que conecta contenedores entre sí, les asigna IP por IPAM y los aísla de los contenedores de otras redes.
2. **¿Qué servicios pueden comunicarse en ella?** Solo los contenedores conectados a esa red (`--network ecommerce-network` o `docker network connect`). Un contenedor en la red `bridge` por defecto o en otra red no puede alcanzarlos por nombre ni, salvo configuración, por IP.
3. **¿Qué relación tiene con la resolución de nombres?** En las redes definidas por el usuario Docker activa su DNS embebido (`127.0.0.11`): cada contenedor puede resolver a los demás por su nombre, sus alias o su nombre de servicio. La red es el ámbito (contexto) del espacio de nombres; en la red `bridge` por defecto esto no ocurre y habría que usar IP o el obsoleto `--link`.

Ejercicio 2 — Crear el servicio de inventario

```
$ docker run -d --name inventory-service --network ecommerce-network nginx
$ docker ps
NAMES               IMAGE     STATUS
inventory-service   nginx     Up Less than a second
$ docker inspect inventory-service
Name=/inventory-service State=running Networks=ecommerce-network IP=172.30.0.2
MAC=f2:2d:e0:e9:c2:d6 DNSNames=[inventory-service d85808269c04]
```

Dato	Valor
Nombre	`inventory-service`
IP	`172.30.0.2`
Red	`ecommerce-network`
Estado	`running`
Identificador	`d85808269c04…` (ID del contenedor)

Diferencia entre nombre y dirección: el nombre lo eligió quien lanzó el contenedor y significa algo (“el servicio de inventario”); la IP la asignó Docker automáticamente y solo sirve para enrutar. Los `DNSNames` muestran que el contenedor tiene dos nombres resolubles: el nombre y su ID corto, que es un identificador.

Ejercicio 3 — Crear un cliente

El PDF usa `docker run -it ... alpine`; el script usa `-d alpine sleep infinity` y `docker exec` para poder automatizarlo. El resultado es el mismo.

```
$ cat /etc/resolv.conf
nameserver 127.0.0.11
$ ping -c 3 inventory-service
PING inventory-service (172.30.0.2): 56 data bytes
64 bytes from 172.30.0.2: seq=0 ttl=64 time=0.066 ms
$ nslookup inventory-service
Server:         127.0.0.11
Address:        127.0.0.11#53
Name:   inventory-service
Address: 172.30.0.2
$ getent hosts inventory-service
172.30.0.2        inventory-service  inventory-service
$ curl http://inventory-service
HTTP 200 desde 172.30.0.2
```

- **¿Quién resolvió el nombre?** El DNS embebido de Docker en `127.0.0.11` (lo indica `Server: 127.0.0.11` y el `resolv.conf` que Docker genera). `getent hosts` muestra que la resolución pasa por la libc del sistema (NSS), que es lo mismo que usa cualquier aplicación.
- **¿Fue necesario conocer la IP?** No. En ningún comando se escribió `172.30.0.2`; se obtuvo a partir del nombre.
- **¿Qué ventaja presenta?** El cliente se configura una sola vez con el nombre; la IP puede cambiar, el servicio se puede recrear o escalar y el cliente sigue funcionando. Además es legible y no depende de la subred de cada máquina.

Ejercicio 4 — Cambio de dirección

Para reproducir el caso del enunciado (`172.18.0.5` → `172.18.0.9`) se lanza un contenedor `filler` antes de recrear inventario: ocupa la IP liberada y obliga a Docker a asignar otra. Sin él, Docker puede devolver la misma IP porque es la primera libre; eso no cambia la conclusión, la IP sigue siendo algo que el cliente no controla.

```
IP de inventory-service ANTES: 172.30.0.2
$ docker stop inventory-service && docker rm inventory-service
$ docker run -d --name filler --network ecommerce-network alpine sleep infinity
$ docker run -d --name inventory-service --network ecommerce-network nginx
IP de inventory-service DESPUES: 172.30.0.4
IP de filler (ocupa la direccion anterior): 172.30.0.2
$ nslookup inventory-service          (desde orders-service)
Name:   inventory-service
Address: 172.30.0.4
$ curl http://inventory-service
HTTP 200 desde 172.30.0.4
$ curl http://172.30.0.2
HTTP 000 -> La IP vieja 172.30.0.2 ya no corresponde a inventory-service
```

1. **¿Cambió la IP?** Sí: de `172.30.0.2` a `172.30.0.4`.
2. **¿Cambió el nombre?** No, sigue siendo `inventory-service`.
3. **¿Fue necesario modificar el cliente?** No. `orders-service` ni se reinició: la siguiente consulta DNS ya devolvió la IP nueva. Quien hubiera usado la IP vieja ahora le habla a `filler`, que ni siquiera es un servidor web.
4. **¿Qué relación tiene con transparencia de ubicación?** Es exactamente eso: el recurso cambió de ubicación y el cliente no se enteró, porque solo conoce el nombre y la plataforma mantiene actualizada la asociación nombre → dirección (transparencia de ubicación y de reubicación).

Ejercicio 5 — DNS tradicional

```
$ nslookup www.google.com
Server:         127.0.0.11
Non-authoritative answer:
Name:   www.google.com   Address: 142.251.157.119
...  (8 direcciones IPv4 y 8 IPv6)
$ dig +noall +answer +stats www.google.com
www.google.com.         299     IN      A       142.251.157.119
...
;; Query time: 4 msec
;; SERVER: 127.0.0.11#53(127.0.0.11) (UDP)
$ dig +noall +answer inventory-service
inventory-service.      600     IN      A       172.30.0.4
$ dig @8.8.8.8 inventory-service
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN
```

Aspecto	DNS de Internet	DNS interno de Docker
Quién responde	Jerarquía raíz → `.com` → `google.com` (autoritativo), vía resolvers recursivos con caché	El daemon de Docker en `127.0.0.11`, que conoce los contenedores de la red
Espacio de nombres	Jerárquico y global (`www.google.com.`)	Plano y local a la red (`inventory-service`)
Respuesta	`Non-authoritative answer`: la da un resolver recursivo con caché, no el servidor autoritativo de `google.com`; TTL 300 s	Autoritativa para la red; TTL 600 s pero siempre con el estado actual
Alcance	Cualquier host de Internet	Solo contenedores de `ecommerce-network`; `8.8.8.8` responde `NXDOMAIN` para `inventory-service`
Varias IP	Sí, para repartir carga y por redundancia	Sí, cuando hay varias instancias del servicio (ejercicio 11)

El DNS embebido hace de *forwarder*: lo que no conoce (`www.google.com`) lo reenvía al DNS del host. Por eso un mismo resolver sirve para nombres internos y externos.

Ejercicio 6 — InventoryService en C#

Se creó la Web API con controladores (`InventoryService/`). Se partió del controlador del enunciado y se agregó un pequeño catálogo y el campo `address` (IP del contenedor) para ver qué instancia atiende cada pedido:

```
[HttpGet("{productId}")]
public IActionResult Get(string productId)
{
    if (!Stock.TryGetValue(productId, out var stock))
        return NotFound(new { Error = $"Product {productId} not found", Server = Environment.MachineName, Addre
ss });

    return Ok(new { ProductId = productId.ToUpperInvariant(), Stock = stock,
                    Server = Environment.MachineName, Address });
}
```

`Environment.MachineName` dentro de un contenedor es su *hostname*, que Docker inicializa con el ID corto del contenedor: un identificador, no un nombre lógico. Por eso en las salidas aparece `server: "86ef2a49f6a5"` mientras que el cliente usa `inventory-service`.

Se ejecuta con `dotnet run` (local) o, como en este repo, dentro de Docker (ejercicio 8).

Ejercicio 7 — OrdersService

`OrdersService/` usa el nombre lógico. En vez de `new HttpClient()` se usa `IHttpClientFactory` (evita agotar sockets) y la URL base viene de configuración:

```
var inventoryBaseUrl = builder.Configuration["Inventory:BaseUrl"] ?? "http://inventory-service:8080";
builder.Services.AddHttpClient("inventory", client => { client.BaseAddress = new Uri(inventoryBaseUrl); ... });

// OrdersController
var response = await _inventory.GetAsync($"api/inventory/{productId}");
var content = await response.Content.ReadAsStringAsync();
_logger.LogInformation("GET {Url} -> {Status} {Content}", url, (int)response.StatusCode, content);
```

Comparación IP directa vs nombre lógico. Se levantaron dos `OrdersService` idénticos: el de Compose con `http://inventory-service:8080` y otro (`orders-by-ip`, puerto 8082) con la IP que tenía inventario en ese momento, `http://172.29.0.2:8080`. Luego se recreó inventario:

```
--- Con ambas configuraciones funcionando ---
:8081 {"requestedUrl":"http://inventory-service:8080/api/inventory/P001","inventory":{"stock":25,"server":"86ef
2a49f6a5","address":"172.29.0.2"}}
:8082 {"requestedUrl":"http://172.29.0.2:8080/api/inventory/P001","inventory":{"stock":25,"server":"86ef2a49f6a
5","address":"172.29.0.2"}}

--- Se recrea inventory-service ---
IP vieja: 172.29.0.2  |  IP nueva: 172.29.0.5  |  filler: 172.29.0.2

:8081 {"requestedUrl":"http://inventory-service:8080/api/inventory/P001","inventory":{"stock":25,"server":"756d
8558c4b8","address":"172.29.0.5"}} [HTTP 200]
:8082 {"requestedUrl":"http://172.29.0.2:8080/api/inventory/P001","error":"Inventory service unreachable: Conne
ction refused (172.29.0.2:8080)"} [HTTP 503]
```

	IP directa	Nombre lógico
Antes de recrear	Funciona	Funciona
Después de recrear	503: la IP ahora es de otro contenedor	Funciona; habla con la instancia nueva
Para arreglarlo	Buscar la IP nueva, cambiar la configuración y reiniciar el cliente	Nada
Varias instancias	Imposible con una sola IP	El nombre resuelve a todas

Ejercicio 8 — Dockerizar los servicios

`InventoryService/Dockerfile` es el del enunciado. El de `OrdersService` es el equivalente (compilación en `sdk:8.0`, ejecución en `aspnet:8.0`):

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet restore
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app/publish .
ENTRYPOINT ["dotnet", "OrdersService.dll"]
```

La imagen `aspnet:8.0` escucha por defecto en el puerto 8080 (`ASPNETCORE_HTTP_PORTS=8080`), por eso la URL es `inventory-service:8080`. Cada proyecto tiene un `.dockerignore` que excluye `bin/` y `obj/`.

Ejercicio 9 — Docker Compose

`docker-compose.yml` es el del enunciado con tres agregados: la variable `Inventory__BaseUrl`, el puerto `8081:8080` de orders para probarlo desde el host, y ningún `container_name` ni puerto publicado en inventario, para poder escalarlo (ejercicio 11).

```
$ docker compose up -d --build
$ docker compose ps
NAME                              SERVICE             STATUS     PORTS
shopresolver-inventory-service-1   inventory-service   Up
shopresolver-orders-service-1      orders-service      Up         0.0.0.0:8081->8080/tcp
$ docker network ls --filter name=ecommerce
0ea1bea35f65   shopresolver_ecommerce-network   bridge    local
Direcciones: inventory-service-1 172.29.0.2 | orders-service-1 172.29.0.3

$ curl localhost:8081/api/orders/resolve
{"name":"inventory-service","addresses":["172.29.0.2"]}
$ curl localhost:8081/api/orders/inventory/P001
{"requestedUrl":"http://inventory-service:8080/api/inventory/P001","inventory":{"productId":"P001","stock":2
5,"server":"86ef2a49f6a5","address":"172.29.0.2"}}
$ curl -X POST localhost:8081/api/orders -d '{"productId":"P001","quantity":2}'
{"id":1,"productId":"P001","quantity":2,"status":"CONFIRMED","servedBy":"86ef2a49f6a5",...}
$ curl -X POST localhost:8081/api/orders -d '{"productId":"P003","quantity":1}'
{"error":"Not enough stock for P003","available":0,"servedBy":"86ef2a49f6a5"} [HTTP 409]
```

Nombres, direcciones y resolución en Compose:

- Compose crea la red con el prefijo del proyecto (`shopresolver_ecommerce-network`) y los contenedores con nombre `<proyecto>-<servicio>-<n>`.
- Además registra el **nombre del servicio** (`inventory-service`) como alias DNS de todos sus contenedores. Ese es el nombre lógico que usa el cliente; el nombre del contenedor es un detalle de implementación.
- Las IP (`.2`, `.3`) las asigna Docker en orden de creación; nadie las configuró.
- `depends_on` solo ordena el arranque; no tiene que ver con la resolución.

Ejercicio 10 — Recrear la infraestructura

```
--- ANTES ---
/shopresolver-inventory-service-1 172.29.0.5
/shopresolver-orders-service-1 172.29.0.3
$ docker compose down     # borra contenedores y red
$ docker compose up -d
--- DESPUES ---
/shopresolver-inventory-service-1 172.29.0.2
/shopresolver-orders-service-1 172.29.0.3
$ curl localhost:8081/api/orders/resolve
{"name":"inventory-service","addresses":["172.29.0.2"]}
$ curl localhost:8081/api/orders/inventory/P002
{"requestedUrl":"http://inventory-service:8080/api/inventory/P002","inventory":{"productId":"P002","stock":8,"s
erver":"ad38a4435513","address":"172.29.0.2"}}
```

Los contenedores son nuevos (otro ID, `server` distinto) e inventario cambió de `172.29.0.5` a `172.29.0.2`. **El cliente no necesitó ninguna modificación**: la imagen y la configuración de `OrdersService` son las mismas y siguen apuntando a `inventory-service:8080`. Vale notar que tras un `down`/`up` limpio las IP suelen repetirse (Docker reparte en orden de creación sobre una red nueva), pero no hay ninguna garantía de eso: depende del orden de arranque, de otros contenedores en la red y de la subred que asigne Docker. Por eso el cliente no debe depender de ellas.

Ejercicio 11 — Múltiples instancias

```
$ docker compose up -d --scale inventory-service=3
inventory-service-1 172.29.0.2 | inventory-service-2 172.29.0.5 | inventory-service-3 172.29.0.4
$ curl localhost:8081/api/orders/resolve
{"name":"inventory-service","addresses":["172.29.0.4","172.29.0.2","172.29.0.5"]}
```

```
--- 12 consultas a http://inventory-service:8080 (server = contenedor que respondió) ---
ad38a4435513 172.29.0.2
080514710eca 172.29.0.5
cfcb185bd2ad 172.29.0.4
ad38a4435513 172.29.0.2
...
--- Reparto de 30 consultas ---
     10 172.29.0.2
     16 172.29.0.4
      4 172.29.0.5
--- Falla una instancia (docker stop inventory-service-1, 172.29.0.2) ---
$ curl localhost:8081/api/orders/resolve
{"name":"inventory-service","addresses":["172.29.0.4","172.29.0.5"]}
     18 172.29.0.4 HTTP 200
     12 172.29.0.5 HTTP 200
```

1. **¿Debería OrdersService conocer las tres IP?** No. Debe conocer un único nombre, `inventory-service`; cuántas instancias hay y dónde están es responsabilidad de la plataforma.
2. **¿Qué problemas generaría?** Configuración que cambia con cada escalado o reinicio; el cliente tendría que implementar su propio balanceo, detectar caídas y reintentar; habría que redesplegarlo para agregar o quitar instancias; las IP obsoletas producen errores o tráfico hacia contenedores equivocados; y se acoplan equipos e infraestructura.
3. **¿Qué mecanismo debería localizar las instancias?** Un servicio de nombres / Service Discovery: aquí el DNS embebido de Docker, que devuelve un registro `A` por cada contenedor del servicio. En Kubernetes, el Service con CoreDNS; en otras plataformas, Consul, Eureka o un balanceador.
4. **¿Cómo se relaciona con balanceo de carga?** El DNS de Docker hace *round-robin DNS*: devuelve las direcciones en distinto orden y cada cliente usa la primera, así la carga se reparte. Para que se note, `OrdersService` abre una conexión nueva por pedido (`ConnectionClose = true`) y entonces resuelve el nombre cada vez; con conexiones persistentes, el cliente se quedaría “pegado” a una instancia. Es un balanceo simple: no sabe de carga ni de salud. En Kubernetes el balanceo lo hace el Service (kube-proxy) detrás de una única IP virtual.
5. **¿Qué ocurre si una instancia falla?** Al detenerse el contenedor, Docker lo quita del DNS: el nombre pasa a resolver solo a las dos instancias restantes y todos los pedidos siguieron respondiendo 200, sin tocar el cliente. Límites: Docker solo lo detecta si el contenedor se detiene (un proceso colgado pero vivo seguiría en el DNS) y las conexiones en curso hacia la instancia caída fallan; por eso en producción se agregan *health checks* y reintentos.

Ejercicio 12 — Service Discovery

1. **¿Qué información mantiene Service Discovery?** Un registro con, por cada nombre lógico de servicio, la lista de instancias disponibles: dirección IP, puerto, estado de salud, y a veces metadatos (versión, zona, pesos, etiquetas) y la fecha del último *heartbeat*.
2. **¿Cómo se registran nuevas instancias?** De dos formas. *Auto-registro*: la instancia, al arrancar, se anota en el registro (Eureka, Consul agent). *Registro por terceros*: la plataforma lo hace sola, que es lo que pasa en Docker (al conectar el contenedor a la red se agrega su nombre y alias) y en Kubernetes (el controlador de EndpointSlices agrega cada Pod que coincide con el `selector` del Service y está *Ready*).
3. **¿Cómo se detectan instancias no disponibles?** Con *health checks* activos (el registro o la plataforma consultan un endpoint como `/health`; en Kubernetes, la `readinessProbe`), con *heartbeats* y TTL (si la

instancia deja de renovar su registro, vence) o por eventos del orquestador (contenedor detenido, Pod eliminado). Las instancias que fallan se quitan de las respuestas.

4. **¿Qué relación existe con nombrado?** Service Discovery *es* resolución de nombres aplicada a servicios dinámicos: el cliente entrega un nombre lógico y recibe direcciones. Agrega a un servicio de nombres clásico el registro automático, la salud y la actualización casi en tiempo real.
5. **¿Por qué es importante en microservicios?** Porque hay muchos servicios, con muchas réplicas efímeras, IP dinámicas y escalado automático. Sin descubrimiento habría que mantener a mano qué instancia está dónde, algo imposible a esa escala. Con él se logran transparencia de ubicación, escalado horizontal, tolerancia a fallas y despliegues sin cortes.

Ejercicio 13 — Kubernetes

`k8s/shopresolver.yaml` despliega el mismo sistema en Kubernetes: un `Deployment` de inventario con 3 réplicas y `readinessProbe` sobre `/health`, un `Service` `inventory-service` (ClusterIP) delante, y `OrdersService` sin ningún cambio, con `Inventory__BaseUrl=http://inventory-service:8080`. El script `scripts/ej13_kubernetes.sh` lo ejecuta en un cluster local de kind (en la salida, `k` es `kubectl -n ecommerce`).

```
$ kubectl get pods -o wide
inventory-service-694dddbb9f-krtgg   1/1  Running  10.244.0.6
inventory-service-694dddbb9f-mb422   1/1  Running  10.244.0.7
inventory-service-694dddbb9f-rwjpw   1/1  Running  10.244.0.5
orders-service-749959f4bd-xjnz6      1/1  Running  10.244.0.8
$ kubectl get service
inventory-service   ClusterIP   10.96.136.163   8080/TCP
orders-service      NodePort    10.96.122.92    8080:30081/TCP
$ kubectl get endpointslices -l kubernetes.io/service-name=inventory-service
inventory-service-kq6v8   IPv4   8080   10.244.0.6,10.244.0.7,10.244.0.5
$ cat /etc/resolv.conf                      (dentro de un Pod)
search ecommerce.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10
$ nslookup inventory-service.ecommerce.svc.cluster.local
Server:   10.96.0.10
Address:  10.96.136.163
$ wget -qO- http://orders-service:8080/api/orders/resolve
{"name":"inventory-service","addresses":["10.96.136.163"]}

--- 30 consultas: el Service reparte entre los Pods ---
     13 inventory-service-694dddbb9f-krtgg 10.244.0.6
      6 inventory-service-694dddbb9f-mb422 10.244.0.7
     11 inventory-service-694dddbb9f-rwjpw 10.244.0.5

$ kubectl delete pod inventory-service-694dddbb9f-krtgg
$ kubectl get pods -l app=inventory-service -o wide
inventory-service-694dddbb9f-kwfvb   1/1  Running  10.244.0.10    <- Pod nuevo, IP nueva
inventory-service-694dddbb9f-mb422   1/1  Running  10.244.0.7
inventory-service-694dddbb9f-rwjpw   1/1  Running  10.244.0.5
$ kubectl get endpointslices ...
inventory-service-kq6v8   IPv4   8080   10.244.0.7,10.244.0.5,10.244.0.10
$ kubectl get service inventory-service
inventory-service   ClusterIP   10.96.136.163   8080/TCP       <- no cambió

      9 inventory-service-694dddbb9f-kwfvb 10.244.0.10
     10 inventory-service-694dddbb9f-mb422 10.244.0.7
     11 inventory-service-694dddbb9f-rwjpw 10.244.0.5
```

1. **¿Qué problema de nombrado resuelve Kubernetes Service?** Da un nombre y una dirección **estables** a un conjunto de Pods efímeros. El `Service` `inventory-service` tiene un nombre DNS (`inventory-service.ecommerce.svc.cluster.local`, resuelto por CoreDNS) y una IP virtual (`10.96.136.163`) que no cambian mientras el Service exista, aunque los Pods de detrás se creen, mueran o se multipliquen. Además selecciona los Pods por etiquetas (`app: inventory-service`), no por dirección, y reparte el tráfico entre ellos.

2. **¿Por qué no conviene referenciar directamente las IP de los Pods?** Porque los Pods son desechables: cada vez que se recrean (falla, despliegue de una versión nueva, reprogramación en otro nodo, escalado) reciben otra IP del rango del cluster. En la práctica, el Pod `…krtgg` (`10.244.0.6`) fue reemplazado por `…kwfvb` (`10.244.0.10`): una referencia a `10.244.0.6` habría quedado rota. Tampoco permitiría repartir carga entre réplicas ni excluir Pods que no están listos.
3. **¿Qué ocurre si un Pod es reemplazado?** El `Deployment` (vía su ReplicaSet) crea uno nuevo para mantener 3 réplicas; el nuevo tiene otro nombre e IP. El controlador de EndpointSlices quita la IP vieja y agrega la nueva cuando el Pod pasa la `readinessProbe`. El Service conserva su nombre y su ClusterIP, y las consultas siguieron repartiéndose entre los 3 Pods actuales.
4. **¿El cliente necesita conocer el cambio?** No. `OrdersService` resuelve `inventory-service` y obtiene siempre la misma ClusterIP; kube-proxy traduce esa IP virtual a la IP de algún Pod listo. El cliente nunca ve las IP de los Pods: ni siquiera tiene que volver a resolver el nombre.
5. **¿Cómo se relaciona con transparencia de ubicación?** Es transparencia de ubicación, migración y replicación al mismo tiempo: el cliente usa un nombre que no dice en qué nodo ni en qué Pod corre el servicio, cuántas réplicas hay ni si fueron reemplazadas. A diferencia de Docker, donde el nombre resuelve directamente a las IP de los contenedores, en Kubernetes hay una indirección más (nombre → IP virtual del Service → Pod), que oculta incluso los cambios de dirección.

Ejercicio 14 — Comparación

Mecanismo	Nombre utilizado	Quién resuelve	Destino	¿Puede cambiar la IP?
DNS Internet	`www.ejemplo.com`	Jerarquía DNS (resolver recursivo → raíz → TLD → autoritativo), con caché	Servidor(es)	Sí, y el cliente lo ve cuando vence el TTL
Docker	`inventory-service`	DNS embebido de Docker (`127.0.0.11`)	Contenedor(es) del servicio, una IP por instancia	Sí, en cada recreación; el DNS se actualiza al instante
Kubernetes	`inventory-service` (`.ecommerce.svc.cluster.local`)	CoreDNS → ClusterIP del Service → kube-proxy elige un Pod	Pods detrás del Service	Las IP de los Pods sí; la ClusterIP del Service no cambia mientras exista
IP directa	`172.18.0.5`	No existe resolución	Host / contenedor que tenga esa IP hoy	La referencia queda obsoleta y puede apuntar a otra entidad

Observado en este trabajo: Docker devolvió una IP distinta por instancia y el cliente reparte resolviendo cada vez; Kubernetes devolvió siempre una sola IP virtual estable y repartió por detrás; la IP directa terminó en 503.

Ejercicio 15 — Conclusión

```
NOMBRE  ──resolución──▶  DIRECCIÓN  ──acceso──▶  ENTIDAD / SERVICIO
inventory-service   DNS de Docker / CoreDNS   172.29.0.4:8080 (hoy)   contenedor o Pod de inventario
```

En un sistema distribuido conviene separar tres cosas: **qué** se quiere usar (el nombre), **dónde** está ahora (la dirección) y **quién** es concretamente (la entidad, con su identificador). La **resolución** es la función que, en el momento del acceso, conecta el nombre con la dirección vigente.

Los componentes de un sistema distribuido moderno cambian todo el tiempo: los contenedores se recrean y reciben otra IP (ejercicios 4 y 10), se escalan a varias instancias (ejercicio 11), fallan y son reemplazados (ejercicios 11 y 13) o se mueven de nodo. Si el cliente guardara direcciones, cada uno de esos eventos lo rompería, como le pasó al `OrdersService` configurado con `172.29.0.2`, que terminó en 503 mientras el que usaba el nombre siguió respondiendo.

Al depender solo del nombre, el cliente queda desacoplado de la ubicación física: la plataforma (DNS de Internet, DNS embebido de Docker, Service y CoreDNS de Kubernetes, o un registro de Service Discovery) se encarga de mantener la asociación nombre → direcciones al día, solo con instancias vivas, y de repartir la carga. Eso es lo que da **transparencia de ubicación, de migración y de replicación**, permite **escalar horizontalmente** y **tolerar fallas** sin modificar ni redesplegar los clientes. Por eso los nombres no son un detalle de configuración sino una pieza central de la arquitectura de cualquier sistema distribuido.

Bibliografía

1. Tanenbaum, A. S. y Van Steen, M. *Distributed Systems*, cap. 6 “Naming”.
2. Coulouris, G., Dollimore, J., Kindberg, T. y Blair, G. *Distributed Systems: Concepts and Design*. Pearson, cap. “Name services”.
3. Docker Docs — *Networking overview* y *Container networking / DNS services*.
4. Microsoft Learn — *ASP.NET Core Web API*, *IHttpClientFactory*, *Containerize a .NET app*.
5. Kubernetes Docs — *Service*, *DNS for Services and Pods*, *EndpointSlices*.

Anexo A — Salidas completas de los scripts

Generadas con los scripts de `scripts/`. Son las salidas reales a partir de las cuales se respondieron los ejercicios.

Ejercicios 1 a 5 — red Docker, DNS embebido, cambio de IP, DNS de Internet

```
===== Ejercicio 1 - Crear una red Docker =====

$ docker network create ecommerce-network
37ddea68d0dc598d6c0e4a334cbd3a718d7c0d835670a59464568c0cbc4cdf9f

$ docker network ls --filter name=ecommerce-network
NETWORK ID     NAME                            DRIVER    SCOPE
37ddea68d0dc   ecommerce-network               bridge    local
8c1e7d6236b2   shopresolver_ecommerce-network   bridge    local

$ docker network inspect ecommerce-network --format Name={{.Name}} Driver={{.Driver}} Subnet={{range .IPAM.Conf
ig}}{{.Subnet}}{{end}} Gateway={{range .IPAM.Config}}{{.Gateway}}{{end}}
Name=ecommerce-network Driver=bridge Subnet=172.30.0.0/16 Gateway=172.30.0.1


===== Ejercicio 2 - Crear el servicio de inventario =====

$ docker run -d --name inventory-service --network ecommerce-network nginx
d85808269c04eea9d5aaf273ac60f581fe1f16a06e1689011de78d6ed2b86b8c

$ docker ps --filter name=inventory-service --format table {{.Names}}\t{{.Image}}\t{{.Status}}
NAMES                              IMAGE                              STATUS
inventory-service                  nginx                              Up Less than a second
shopresolver-inventory-service-1   shopresolver-inventory-service   Up 10 seconds

$ docker inspect inventory-service --format Name={{.Name}} State={{.State.Status}} Networks={{range $k, $v := .
NetworkSettings.Networks}}{{$k}} IP={{$v.IPAddress}} MAC={{$v.MacAddress}} DNSNames={{$v.DNSNames}}{{end}}
Name=/inventory-service State=running Networks=ecommerce-network IP=172.30.0.2 MAC=f2:2d:e0:e9:c2:d6 DNSNames=
[inventory-service d85808269c04]


===== Ejercicio 3 - Crear un cliente =====

$ docker run -d --name orders-service --network ecommerce-network alpine sleep infinity
28d00ef466270f66c853befd974860f5b809fd41bed24f787cdf3733a4e672f1

$ docker exec orders-service cat /etc/resolv.conf
# Generated by Docker Engine.
# This file can be edited; Docker Engine will not make further changes once it
# has been modified.

nameserver 127.0.0.11
options ndots:0

# Based on host file: '/etc/resolv.conf' (internal resolver)
# ExtServers: [host(192.168.65.7)]
# Overrides: []
# Option ndots from: internal

$ docker exec orders-service ping -c 3 inventory-service
PING inventory-service (172.30.0.2): 56 data bytes
64 bytes from 172.30.0.2: seq=0 ttl=64 time=0.080 ms
64 bytes from 172.30.0.2: seq=1 ttl=64 time=0.082 ms
64 bytes from 172.30.0.2: seq=2 ttl=64 time=0.079 ms

--- inventory-service ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
```

```
round-trip min/avg/max = 0.079/0.080/0.082 ms

$ docker exec orders-service nslookup inventory-service
Server:         127.0.0.11
Address:        127.0.0.11#53

Non-authoritative answer:
Name:   inventory-service
Address: 172.30.0.2


$ docker exec orders-service getent hosts inventory-service
172.30.0.2        inventory-service  inventory-service

$ docker exec orders-service sh -c curl -s -o /dev/null -w 'HTTP %{http_code} desde %{remote_ip}\n' http://inve
ntory-service
HTTP 200 desde 172.30.0.2


===== Ejercicio 4 - Cambio de direccion =====

IP de inventory-service ANTES: 172.30.0.2
$ docker stop inventory-service
inventory-service

$ docker rm inventory-service
inventory-service

$ docker run -d --name filler --network ecommerce-network alpine sleep infinity
7b9c2bda1753b582a1450d8660dbe8f3b9399b95077bee942e30fc4547588b03

$ docker run -d --name inventory-service --network ecommerce-network nginx
c6e733bb1602a26923a2af7ab777c0da9b6a4ceeb13802a0a4e69409a030a56d

$ docker inspect inventory-service --format Name={{.Name}} State={{.State.Status}} IP={{range .NetworkSettings.
Networks}}{{.IPAddress}}{{end}}
Name=/inventory-service State=running IP=172.30.0.4

IP de inventory-service DESPUES: 172.30.0.4
IP de filler (ocupa la direccion anterior): 172.30.0.2

$ docker exec orders-service nslookup inventory-service
Server:         127.0.0.11
Address:        127.0.0.11#53

Non-authoritative answer:
Name:   inventory-service
Address: 172.30.0.4


$ docker exec orders-service sh -c curl -s -o /dev/null -w 'HTTP %{http_code} desde %{remote_ip}\n' http://inve
ntory-service
HTTP 200 desde 172.30.0.4

$ docker exec orders-service sh -c curl -s -m 3 -o /dev/null -w 'HTTP %{http_code} desde %{remote_ip}\n' htt
p://172.30.0.2 || echo 'La IP vieja 172.30.0.2 ya no corresponde a inventory-service'
HTTP 000 desde
La IP vieja 172.30.0.2 ya no corresponde a inventory-service


===== Ejercicio 5 - DNS tradicional =====

$ docker exec orders-service nslookup www.google.com
Server:         127.0.0.11
Address:        127.0.0.11#53

Non-authoritative answer:
Name:   www.google.com
Address: 142.251.157.119
Name:   www.google.com
Address: 142.251.152.119
Name:   www.google.com
Address: 142.251.151.119
```

```
Name:   www.google.com
Address: 142.251.150.119
Name:   www.google.com
Address: 142.251.153.119
Name:   www.google.com
Address: 142.251.156.119
Name:   www.google.com
Address: 142.251.154.119
Name:   www.google.com
Address: 142.251.155.119
Name:   www.google.com
Address: 2001:4860:4829:7700::
Name:   www.google.com
Address: 2001:4860:4826:7700::
Name:   www.google.com
Address: 2001:4860:4828:7700::
Name:   www.google.com
Address: 2001:4860:4827:7700::
Name:   www.google.com
Address: 2001:4860:482a:7700::
Name:   www.google.com
Address: 2001:4860:482c:7700::
Name:   www.google.com
Address: 2001:4860:482b:7700::
Name:   www.google.com
Address: 2001:4860:482d:7700::


$ docker exec orders-service dig +noall +answer +stats www.google.com
www.google.com.         299     IN      A       142.251.157.119
www.google.com.         299     IN      A       142.251.152.119
www.google.com.         299     IN      A       142.251.151.119
www.google.com.         299     IN      A       142.251.150.119
www.google.com.         299     IN      A       142.251.153.119
www.google.com.         299     IN      A       142.251.156.119
www.google.com.         299     IN      A       142.251.154.119
www.google.com.         299     IN      A       142.251.155.119
;; Query time: 4 msec
;; SERVER: 127.0.0.11#53(127.0.0.11) (UDP)
;; WHEN: Wed Nov 11 19:49:20 UTC 2026
;; MSG SIZE  rcvd: 160


$ docker exec orders-service dig +noall +answer inventory-service
inventory-service.      600     IN      A       172.30.0.4

$ docker exec orders-service sh -c dig @8.8.8.8 inventory-service +noall +comments | grep status
;; ->>HEADER<<- opcode: QUERY, status: NXDOMAIN, id: 39826
```

Ejercicios 7, 9, 10 y 11 — servicios C# con Docker Compose

```
===== Ejercicio 9 - Docker Compose =====

$ docker compose up -d
 Network shopresolver_ecommerce-network Creating
 Network shopresolver_ecommerce-network Created
 Container shopresolver-inventory-service-1 Creating
 Container shopresolver-inventory-service-1 Created
 Container shopresolver-orders-service-1 Creating
 Container shopresolver-orders-service-1 Created
 Container shopresolver-inventory-service-1 Starting
 Container shopresolver-inventory-service-1 Started
 Container shopresolver-orders-service-1 Starting
 Container shopresolver-orders-service-1 Started

$ docker compose ps --format table {{.Name}}\t{{.Service}}\t{{.Status}}\t{{.Ports}}
NAME                              SERVICE             STATUS         PORTS
shopresolver-inventory-service-1   inventory-service   Up 2 seconds
shopresolver-orders-service-1      orders-service      Up 1 second    0.0.0.0:8081->8080/tcp, [::]:8081->8080/t
```

```
cp

$ docker network ls --filter name=ecommerce
NETWORK ID     NAME                              DRIVER    SCOPE
0ea1bea35f65   shopresolver_ecommerce-network   bridge    local

Direcciones asignadas:
/shopresolver-inventory-service-1 172.29.0.2
/shopresolver-orders-service-1 172.29.0.3

$ curl -s http://localhost:8081/api/orders/resolve
{"name":"inventory-service","addresses":["172.29.0.2"]}

$ curl -s http://localhost:8081/api/orders/inventory/P001
{"requestedUrl":"http://inventory-service:8080/api/inventory/P001","inventory":{"productId":"P001","stock":2
5,"server":"86ef2a49f6a5","address":"172.29.0.2"}}

$ curl -s -X POST http://localhost:8081/api/orders -H content-type: application/json -d {"productId":"P001","qu
antity":2}
{"id":1,"productId":"P001","quantity":2,"status":"CONFIRMED","servedBy":"86ef2a49f6a5","createdAt":"2026-11-11T
19:48:11.0314232Z"}

$ curl -s -w  [HTTP %{http_code}] -X POST http://localhost:8081/api/orders -H content-type: application/json -d
{"productId":"P003","quantity":1}
{"error":"Not enough stock for P003","available":0,"servedBy":"86ef2a49f6a5"} [HTTP 409]

$ docker compose logs --no-log-prefix orders-service --tail 4
info: Microsoft.AspNetCore.Routing.EndpointMiddleware[1]
      Executed endpoint 'OrdersService.Controllers.OrdersController.Create (OrdersService)'
info: Microsoft.AspNetCore.Hosting.Diagnostics[2]
      Request finished HTTP/1.1 POST http://localhost:8081/api/orders - 409 - application/json;+charset=utf-8
8.3326ms


===== Ejercicio 7 - IP directa vs nombre logico =====

Se levanta un segundo OrdersService (orders-by-ip) configurado con la IP fija http://172.29.0.2:8080
$ docker run -d --name orders-by-ip --network shopresolver_ecommerce-network -p 8082:8080 -e Inventory__BaseUrl
=http://172.29.0.2:8080 shopresolver-orders-service
50f86a68fc46bd1685931ea622326f57552acdaf38cfc8e4b89b7c8995b94210

--- Con ambas configuraciones funcionando ---
$ curl -s http://localhost:8081/api/orders/inventory/P001
{"requestedUrl":"http://inventory-service:8080/api/inventory/P001","inventory":{"productId":"P001","stock":2
5,"server":"86ef2a49f6a5","address":"172.29.0.2"}}

$ curl -s http://localhost:8082/api/orders/inventory/P001
{"requestedUrl":"http://172.29.0.2:8080/api/inventory/P001","inventory":{"productId":"P001","stock":25,"serve
r":"86ef2a49f6a5","address":"172.29.0.2"}}

--- Se recrea inventory-service (filler ocupa la IP liberada, como en el caso 172.18.0.5 -> 172.18.0.9) ---
$ docker compose stop inventory-service
 Container shopresolver-inventory-service-1 Stopping
 Container shopresolver-inventory-service-1 Stopped

$ docker compose rm -f inventory-service
Going to remove shopresolver-inventory-service-1
 Container shopresolver-inventory-service-1 Removing
 Container shopresolver-inventory-service-1 Removed

$ docker run -d --name filler --network shopresolver_ecommerce-network alpine sleep infinity
7c0136da501cbda362ab2c0ce3e85644377ccd0a83ffd6700cc48766dde95042

$ docker compose up -d inventory-service
 Container shopresolver-inventory-service-1 Creating
 Container shopresolver-inventory-service-1 Created
 Container shopresolver-inventory-service-1 Starting
 Container shopresolver-inventory-service-1 Started

IP vieja: 172.29.0.2  |  IP nueva: 172.29.0.5  |  filler: 172.29.0.2

$ curl -s -w  [HTTP %{http_code}] http://localhost:8081/api/orders/inventory/P001
{"requestedUrl":"http://inventory-service:8080/api/inventory/P001","inventory":{"productId":"P001","stock":2
```

```
5,"server":"756d8558c4b8","address":"172.29.0.5"}} [HTTP 200]

$ curl -s -w  [HTTP %{http_code}] http://localhost:8082/api/orders/inventory/P001
{"requestedUrl":"http://172.29.0.2:8080/api/inventory/P001","error":"Inventory service unreachable: Connection
refused (172.29.0.2:8080)"} [HTTP 503]


===== Ejercicio 10 - Recrear la infraestructura =====

--- ANTES ---
/shopresolver-inventory-service-1 172.29.0.5
/shopresolver-orders-service-1 172.29.0.3
$ docker network inspect shopresolver_ecommerce-network --format Subnet={{range .IPAM.Config}}{{.Subnet}}{{en
d}}
Subnet=172.29.0.0/16

$ docker compose down
 Container shopresolver-orders-service-1 Stopping
 Container shopresolver-orders-service-1 Stopped
 Container shopresolver-orders-service-1 Removing
 Container shopresolver-orders-service-1 Removed
 Container shopresolver-inventory-service-1 Stopping
 Container shopresolver-inventory-service-1 Stopped
 Container shopresolver-inventory-service-1 Removing
 Container shopresolver-inventory-service-1 Removed
 Network shopresolver_ecommerce-network Removing
 Network shopresolver_ecommerce-network Removed

$ docker compose up -d
 Network shopresolver_ecommerce-network Creating
 Network shopresolver_ecommerce-network Created
 Container shopresolver-inventory-service-1 Creating
 Container shopresolver-inventory-service-1 Created
 Container shopresolver-orders-service-1 Creating
 Container shopresolver-orders-service-1 Created
 Container shopresolver-inventory-service-1 Starting
 Container shopresolver-inventory-service-1 Started
 Container shopresolver-orders-service-1 Starting
 Container shopresolver-orders-service-1 Started

--- DESPUES ---
/shopresolver-inventory-service-1 172.29.0.2
/shopresolver-orders-service-1 172.29.0.3
$ docker network inspect shopresolver_ecommerce-network --format Subnet={{range .IPAM.Config}}{{.Subnet}}{{en
d}}
Subnet=172.29.0.0/16

$ curl -s http://localhost:8081/api/orders/resolve
{"name":"inventory-service","addresses":["172.29.0.2"]}

$ curl -s http://localhost:8081/api/orders/inventory/P002
{"requestedUrl":"http://inventory-service:8080/api/inventory/P002","inventory":{"productId":"P002","stock":8,"s
erver":"ad38a4435513","address":"172.29.0.2"}}


===== Ejercicio 11 - Multiples instancias =====

$ docker compose up -d --scale inventory-service=3
 Container shopresolver-inventory-service-1 Running
 Container shopresolver-orders-service-1 Running
 Container shopresolver-inventory-service-3 Creating
 Container shopresolver-inventory-service-2 Creating
 Container shopresolver-inventory-service-2 Created
 Container shopresolver-inventory-service-3 Created
 Container shopresolver-inventory-service-3 Starting
 Container shopresolver-inventory-service-3 Started
 Container shopresolver-inventory-service-2 Starting
 Container shopresolver-inventory-service-2 Started

$ docker compose ps --format table {{.Name}}\t{{.Service}}\t{{.Status}}
NAME                              SERVICE             STATUS
shopresolver-inventory-service-1   inventory-service   Up 11 seconds
shopresolver-inventory-service-2   inventory-service   Up 4 seconds
```

```
shopresolver-inventory-service-3   inventory-service   Up 5 seconds
shopresolver-orders-service-1      orders-service      Up 10 seconds

Direcciones asignadas:
/shopresolver-inventory-service-1 172.29.0.2
/shopresolver-inventory-service-2 172.29.0.5
/shopresolver-inventory-service-3 172.29.0.4
/shopresolver-orders-service-1 172.29.0.3

$ curl -s http://localhost:8081/api/orders/resolve
{"name":"inventory-service","addresses":["172.29.0.4","172.29.0.2","172.29.0.5"]}

--- 12 consultas al mismo nombre http://inventory-service:8080 ---
ad38a4435513 172.29.0.2
080514710eca 172.29.0.5
cfcb185bd2ad 172.29.0.4
ad38a4435513 172.29.0.2
ad38a4435513 172.29.0.2
cfcb185bd2ad 172.29.0.4
cfcb185bd2ad 172.29.0.4
080514710eca 172.29.0.5
cfcb185bd2ad 172.29.0.4
cfcb185bd2ad 172.29.0.4
080514710eca 172.29.0.5
cfcb185bd2ad 172.29.0.4

--- Reparto de las consultas por instancia ---
     10 172.29.0.2
     16 172.29.0.4
      4 172.29.0.5

--- Falla una instancia ---
$ docker stop ad38a4435513ea75f64d1a5cb11621adab989e11a451cf7555cc02618f9dd0ec
ad38a4435513ea75f64d1a5cb11621adab989e11a451cf7555cc02618f9dd0ec

$ curl -s http://localhost:8081/api/orders/resolve
{"name":"inventory-service","addresses":["172.29.0.4","172.29.0.5"]}

     18 172.29.0.4 HTTP 200
     12 172.29.0.5 HTTP 200

$ docker compose up -d --scale inventory-service=1
 Container shopresolver-orders-service-1 Running
 Container shopresolver-inventory-service-2 Stopping
 Container shopresolver-inventory-service-3 Stopping
 Container shopresolver-inventory-service-2 Stopped
 Container shopresolver-inventory-service-2 Removing
 Container shopresolver-inventory-service-2 Removed
 Container shopresolver-inventory-service-3 Stopped
 Container shopresolver-inventory-service-3 Removing
 Container shopresolver-inventory-service-3 Removed
 Container shopresolver-inventory-service-1 Starting
 Container shopresolver-inventory-service-1 Started

Salida guardada en /c/Users/Jose/Desktop/j/FACU/SO/2do/shopresolver/docs/salidas/ej07-11.txt
```

Ejercicio 13 — Kubernetes (cluster kind)

```
$ kubectl --context kind-shopresolver apply -f k8s/shopresolver.yaml
namespace/ecommerce created
deployment.apps/inventory-service created
service/inventory-service created
deployment.apps/orders-service created
service/orders-service created

===== Ejercicio 13 - Kubernetes =====

$ k get pods -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP           NODE
NOMINATED NODE   READINESS GATES
```

```
client                                1/1     Running   0          7s    10.244.0.9   shopresolver-control-plane
<none>           <none>
inventory-service-694dddbb9f-krtgg    1/1     Running   0          9s    10.244.0.6   shopresolver-control-plane
<none>           <none>
inventory-service-694dddbb9f-mb422    1/1     Running   0          9s    10.244.0.7   shopresolver-control-plane
<none>           <none>
inventory-service-694dddbb9f-rwjpw    1/1     Running   0          9s    10.244.0.5   shopresolver-control-plane
<none>           <none>
orders-service-749959f4bd-xjnz6       1/1     Running   0          9s    10.244.0.8   shopresolver-control-plane
<none>           <none>

$ k get service
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
inventory-service   ClusterIP   10.96.136.163   <none>        8080/TCP         9s
orders-service      NodePort    10.96.122.92    <none>        8080:30081/TCP   9s

$ k get endpointslices -l kubernetes.io/service-name=inventory-service
NAME                        ADDRESSTYPE   PORTS   ENDPOINTS                           AGE
inventory-service-kq6v8   IPv4          8080    10.244.0.6,10.244.0.7,10.244.0.5   9s

$ k exec client -- cat /etc/resolv.conf
search ecommerce.svc.cluster.local svc.cluster.local cluster.local
nameserver 10.96.0.10
options ndots:5

$ k exec client -- nslookup inventory-service.ecommerce.svc.cluster.local
Server:         10.96.0.10
Address:        10.96.0.10:53


Name:   inventory-service.ecommerce.svc.cluster.local
Address: 10.96.136.163


--- OrdersService resuelve el nombre: obtiene la ClusterIP del Service, no las IP de los Pods ---
$ k exec client -- wget -qO- http://orders-service:8080/api/orders/resolve
{"name":"inventory-service","addresses":["10.96.136.163"]}

--- 30 consultas: el Service reparte entre los Pods ---
     13 inventory-service-694dddbb9f-krtgg 10.244.0.6
      6 inventory-service-694dddbb9f-mb422 10.244.0.7
     11 inventory-service-694dddbb9f-rwjpw 10.244.0.5

--- Se elimina el Pod inventory-service-694dddbb9f-krtgg; el Deployment crea uno nuevo con otra IP ---
$ k delete pod inventory-service-694dddbb9f-krtgg
pod "inventory-service-694dddbb9f-krtgg" deleted from ecommerce namespace

$ k get pods -l app=inventory-service -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE
NOMINATED NODE   READINESS GATES
inventory-service-694dddbb9f-kwfvb   1/1     Running   0          13s   10.244.0.10   shopresolver-control-plan
e   <none>           <none>
inventory-service-694dddbb9f-mb422   1/1     Running   0          31s   10.244.0.7    shopresolver-control-plan
e   <none>           <none>
inventory-service-694dddbb9f-rwjpw   1/1     Running   0          31s   10.244.0.5    shopresolver-control-plan
e   <none>           <none>

$ k get endpointslices -l kubernetes.io/service-name=inventory-service
NAME                        ADDRESSTYPE   PORTS   ENDPOINTS                           AGE
inventory-service-kq6v8   IPv4          8080    10.244.0.7,10.244.0.5,10.244.0.10   31s

$ k get service inventory-service
NAME                TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
inventory-service   ClusterIP   10.96.136.163   <none>        8080/TCP   31s

      9 inventory-service-694dddbb9f-kwfvb 10.244.0.10
     10 inventory-service-694dddbb9f-mb422 10.244.0.7
     11 inventory-service-694dddbb9f-rwjpw 10.244.0.5

Para borrar el cluster: kind delete cluster --name shopresolver
```

Anexo B — Código fuente principal

InventoryService/Controllers/InventoryController.cs

```
using System.Net;
using System.Net.Sockets;
using Microsoft.AspNetCore.Mvc;

namespace InventoryService.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class InventoryController : ControllerBase
    {
        private static readonly Dictionary<string, int> Stock = new(StringComparer.OrdinalIgnoreCase)
        {
            ["P001"] = 25,
            ["P002"] = 8,
            ["P003"] = 0,
        };

        // Container IP, so the client can see which address answered behind the logical name.
        private static readonly string Address = Dns.GetHostAddresses(Dns.GetHostName())
            .FirstOrDefault(a => a.AddressFamily == AddressFamily.InterNetwork)?.ToString() ?? "unknown";

        [HttpGet]
        public IActionResult GetAll()
        {
            return Ok(new
            {
                Products = Stock.Select(p => new { ProductId = p.Key, Stock = p.Value }),
                Server = Environment.MachineName,
                Address,
            });
        }

        [HttpGet("{productId}")]
        public IActionResult Get(string productId)
        {
            if (!Stock.TryGetValue(productId, out var stock))
            {
                return NotFound(new { Error = $"Product {productId} not found", Server = Environment.MachineNam
e, Address });
            }

            return Ok(new
            {
                ProductId = productId.ToUpperInvariant(),
                Stock = stock,
                Server = Environment.MachineName,
                Address,
            });
        }
    }
}
```

InventoryService/Program.cs

```
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();

var app = builder.Build();

app.MapControllers();
```

```
app.MapGet("/health", () => Results.Ok(new { Status = "UP", Server = Environment.MachineName }));

app.Run();
```

OrdersService/Controllers/OrdersController.cs

```
using System.Collections.Concurrent;
using System.Net;
using System.Net.Http.Json;
using System.Text.Json;
using Microsoft.AspNetCore.Mvc;

namespace OrdersService.Controllers
{
    public record CreateOrderRequest(string ProductId, int Quantity);

    public record Order(int Id, string ProductId, int Quantity, string Status, string ServedBy, DateTime Create
dAt);

    [ApiController]
    [Route("api/[controller]")]
    public class OrdersController : ControllerBase
    {
        private static readonly ConcurrentDictionary<int, Order> Orders = new();
        private static int _nextId;

        private readonly HttpClient _inventory;
        private readonly ILogger<OrdersController> _logger;

        public OrdersController(IHttpClientFactory httpClientFactory, ILogger<OrdersController> logger)
        {
            _inventory = httpClientFactory.CreateClient("inventory");
            _logger = logger;
        }

        // Exercise 7: the client only knows the logical name inventory-service.
        [HttpGet("inventory/{productId}")]
        public async Task<IActionResult> GetInventory(string productId)
        {
            var url = new Uri(_inventory.BaseAddress!, $"api/inventory/{productId}");
            try
            {
                var response = await _inventory.GetAsync(url);
                var content = await response.Content.ReadAsStringAsync();
                _logger.LogInformation("GET {Url} -> {Status} {Content}", url, (int)response.StatusCode, conten
t);

                return StatusCode((int)response.StatusCode, new
                {
                    RequestedUrl = url.ToString(),
                    Inventory = JsonSerializer.Deserialize<JsonElement>(content),
                });
            }
            catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException)
            {
                _logger.LogWarning("GET {Url} failed: {Error}", url, ex.Message);
                return StatusCode(StatusCodes.Status503ServiceUnavailable, new
                {
                    RequestedUrl = url.ToString(),
                    Error = $"Inventory service unreachable: {ex.Message}",
                });
            }
        }

        // Shows how the name is resolved from inside the container (Docker / Kubernetes DNS).
        [HttpGet("resolve")]
        public async Task<IActionResult> Resolve([FromQuery] string? host)
        {
            var name = host ?? _inventory.BaseAddress!.Host;
            try
```

```
            {
                var addresses = await Dns.GetHostAddressesAsync(name);
                return Ok(new { Name = name, Addresses = addresses.Select(a => a.ToString()) });
            }
            catch (Exception ex)
            {
                return NotFound(new { Name = name, Error = ex.Message });
            }
        }

        [HttpGet]
        public IActionResult GetAll() => Ok(Orders.Values.OrderBy(o => o.Id));

        [HttpPost]
        public async Task<IActionResult> Create(CreateOrderRequest request)
        {
            if (request.Quantity <= 0)
            {
                return BadRequest(new { Error = "Quantity must be greater than zero" });
            }

            HttpResponseMessage response;
            try
            {
                response = await _inventory.GetAsync($"api/inventory/{request.ProductId}");
            }
            catch (Exception ex) when (ex is HttpRequestException or TaskCanceledException)
            {
                return StatusCode(StatusCodes.Status503ServiceUnavailable, new { Error = $"Inventory service un
reachable: {ex.Message}" });
            }

            if (response.StatusCode == HttpStatusCode.NotFound)
            {
                return UnprocessableEntity(new { Error = $"Product {request.ProductId} does not exist" });
            }

            var stock = await response.Content.ReadFromJsonAsync<JsonElement>();
            var available = stock.GetProperty("stock").GetInt32();
            var servedBy = stock.GetProperty("server").GetString() ?? "unknown";

            if (available < request.Quantity)
            {
                return Conflict(new { Error = $"Not enough stock for {request.ProductId}", Available = availabl
e, ServedBy = servedBy });
            }

            var id = Interlocked.Increment(ref _nextId);
            var order = new Order(id, request.ProductId.ToUpperInvariant(), request.Quantity, "CONFIRMED", serv
edBy, DateTime.UtcNow);
            Orders[id] = order;

            return Created($"/api/orders/{id}", order);
        }
    }
}
```

OrdersService/Program.cs

```
var builder = WebApplication.CreateBuilder(args);

// Logical name of the inventory service. It can be overridden with the
// environment variable Inventory__BaseUrl (e.g. to compare against a fixed IP).
var inventoryBaseUrl = builder.Configuration["Inventory:BaseUrl"] ?? "http://inventory-service:8080";

builder.Services.AddControllers();
builder.Services.AddHttpClient("inventory", client =>
{
    client.BaseAddress = new Uri(inventoryBaseUrl);
    client.Timeout = TimeSpan.FromSeconds(3);
```

```
    // Do not reuse TCP connections: every request opens a new one, so the name is
    // resolved again and the requests are spread across all the instances
    // returned by the DNS (exercise 11).
    client.DefaultRequestHeaders.ConnectionClose = true;
});

var app = builder.Build();

app.MapControllers();
app.MapGet("/health", () => Results.Ok(new { Status = "UP", Server = Environment.MachineName }));

app.Logger.LogInformation("OrdersService will reach inventory at {InventoryBaseUrl}", inventoryBaseUrl);

app.Run();
```

OrdersService/Dockerfile

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet restore
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /app/publish .
ENTRYPOINT ["dotnet", "OrdersService.dll"]
```

docker-compose.yml

```
services:
  inventory-service:
    build: ./InventoryService
    # No container_name and no host port: this allows
    # docker compose up -d --scale inventory-service=3 (exercise 11).
    networks:
      - ecommerce-network

  orders-service:
    build: ./OrdersService
    depends_on:
      - inventory-service
    environment:
      Inventory__BaseUrl: http://inventory-service:8080
    ports:
      - "8081:8080"
    networks:
      - ecommerce-network

networks:
  ecommerce-network:
    driver: bridge
```

k8s/shopresolver.yaml

```
# Exercise 13: the same platform on Kubernetes.
# inventory-service runs as 3 Pods behind a Service with a stable name;
# OrdersService keeps using http://inventory-service:8080 without changes.
#
#   docker compose build
#   kubectl apply -f k8s/shopresolver.yaml
#
# imagePullPolicy: Never uses the images built locally (Docker Desktop
# Kubernetes shares the image store with Docker).
apiVersion: v1
```

```
kind: Namespace
metadata:
  name: ecommerce
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: inventory-service
  namespace: ecommerce
spec:
  replicas: 3
  selector:
    matchLabels:
      app: inventory-service
  template:
    metadata:
      labels:
        app: inventory-service
    spec:
      containers:
        - name: inventory
          image: shopresolver-inventory-service:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 8080
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: inventory-service
  namespace: ecommerce
spec:
  selector:
    app: inventory-service
  ports:
    - port: 8080
      targetPort: 8080
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: orders-service
  namespace: ecommerce
spec:
  replicas: 1
  selector:
    matchLabels:
      app: orders-service
  template:
    metadata:
      labels:
        app: orders-service
    spec:
      containers:
        - name: orders
          image: shopresolver-orders-service:latest
          imagePullPolicy: Never
          env:
            - name: Inventory__BaseUrl
              value: http://inventory-service:8080
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: orders-service
  namespace: ecommerce
```

```
spec:
  type: NodePort
  selector:
    app: orders-service
  ports:
    - port: 8080
      targetPort: 8080
      nodePort: 30081
```